

Preserving Historical Texts: A Pipeline for Converting old german job description documents to TEI XML

Thesis Technical Documentation

submitted by

Sai venkata sahith reddy Kadapaiahgari

First supervisor: Prof. Dr. Frank Hopfgartner
Institute for Web Science and Technologies

Second supervisor: Dr. Jens Dörpinghaus
Institute for Web Science and Technologies

Koblenz, July 2025

Abstract

The Federal Institute for Vocational Training (BIBB) houses a substantial archive of historical German Vocational Education and Training (VET) and Continuing Vocational Education and Training (CVET) documents. While these documents offer rich insights into the evolution of vocational education in Germany, they remain largely inaccessible due to their format as scanned paper files, lacking structured, machine-readable content. This research addresses this accessibility issue by designing and implementing a complete workflow to extract text from scanned images and convert it into TEI (Text Encoding Initiative) XML, thereby preserving both content and structural metadata. The workflow developed integrates Optical Character Recognition (OCR) techniques to extract text from scanned documents, followed by a custom rule-based system for converting unstructured text into structured TEI XML. A particular focus was placed on overcoming the challenges associated with OCR inaccuracies, such as character misrecognition and inconsistent formatting, especially when dealing with historical print styles and document degradation. In this study, research questions are (1) How can a pipeline be developed to transform scanned GDR job description documents, specifically in the harmonica manufacturing domain, into structured TEI XML format?, (2) What are the primary challenges in accurately extracting text and how to solve them by preserving hierarchical structure?, (3) How can we use the deep learning techniques or rule-based techniques to identify the sections like headers, tables, etc accurately for complex document layouts?. The study involved evaluating different OCR engines and refining parsing rules iteratively to ensure semantic accuracy and structural fidelity in the generated TEI XML. Results show a significant improvement in the quality and usability of the converted documents, enabling better searchability, archiving, and potential future analysis.

Contents

1 System Setup 1

1.1 Installation of PaddlePaddle 1

1.2 Installation of PaddleX 2

1.2.1 Wheel Package Installation Mode 2

1.2.2 Plugin Installation Mode 2

1.2.3 Installing PaddleX on Linux(using Docker) 3

2 Model Integration 4

2.1 Downloading the trained model into local machine 4

2.2 Running the Pipeline 5

2.2.1 First Step: 5

2.2.2 Second Step: 6

2.2.3 Third Step: 8

3 Conclusion 9

1 System Setup

To carry out this research, the first step is to configure a few items on the local machine and pre-process the data. Let us dive into those steps more deeply.

1.1 Installation of PaddlePaddle

The first step of installation in this process is to install PaddlePaddle, this can be done in two ways, via **pip** or **docker**[1].

To install via pip, please refer to the following commands to install PaddlePaddle in your current environment using pip[1]:

For CPU:

```
python -m pip install paddlepaddle==3.0.0 -i
https://www.paddlepaddle.org.cn/packages/stable/cpu/
```

For GPU, requires a GPU driver version greater than 450.80.02 (Linux) or 452.39 (Windows)[1].

```
python -m pip install paddlepaddle-gpu==3.0.0 -i
https://www.paddlepaddle.org.cn/packages/stable/cu118/
```

To install via Docker, the docker version should be greater than **19.03** please refer to the following commands to use the official Docker image of the PaddlePaddle framework to create a container named paddlex and map the current working directory to the /paddle directory inside the container[1]:

For CPU:

```
docker run --name paddlex -v $PWD:/paddle --shm-size=8G
--network=host -it ccr-2vdh3abv-pub.cnc.bj.baidubce.com/
paddlepaddle/paddle:3.0.0 /bin/bash
```

For GPU, requires a GPU driver version greater than 450.80.02 (Linux) or 452.39 (Windows)[1].

```
docker run --gpus all --name paddlex -v $PWD:/paddle
--shm-size=8G --network=host -it ccr-2vdh3abv-pub.cnc.bj.
baidubce.com/
paddlepaddle/paddle:3.0.0-gpu-cuda11.8-cudnn8.9-trt8.6 /bin/bash
```

After installation, verify that the paddle paddle is installed successfully or not using the following command[1].

```
python -c "import_paddle;_print(paddle.__version__)"
```

If the installation is successful, the following content will be the output.

```
3.0.0-rc0
```

With this our first library PaddlePaddle is installed. Now we have to install PaddleX, which contains many deep learning models that can be used for optical character recognition.

1.2 Installation of PaddleX

To seek higher accuracy from existing models, use PaddleX's custom development capabilities to develop better structure analysis models [2]. Before developing a structure analysis model with PaddleX, ensure that you have installed PaddleX's Detection-related model training capabilities[2].

PaddleX offers two installation modes: Installation of the wheel package and installation of the plug-ins. In the following, we introduce their respective application scenarios:

1.2.1 Wheel Package Installation Mode

If the use case for PaddleX involves model inference and integration, a more convenient and lightweight wheel package installation mode is recommended. After installing PaddlePaddle, install the paddleX wheel package by executing the following commands[2].

```
pip install paddlex==3.0.0
```

To install all the dependencies required for the basic features of PaddleX, use the following command[2].

```
pip install "paddlex[base]==3.0.1"
```

To install the dependencies required for specific features, use the command below[2]

```
pip install "paddlex[ocr]==3.0.1"
```

1.2.2 Plugin Installation Mode

If the use case for PaddleX involves custom development (e.g., retraining models, fine-tuning models, customizing model structures, customizing inference codes, etc.), the more powerful plugin installation mode is recommended [2].

After installing the PaddleX plugins that are needed, you can not only perform inference and integration with the supported models, but also perform advanced operations such as model training for custom development [2].

To check the PaddleX-supported model training plugins refer to [2]. Determine the name(s) of the plugin(s) that are needed based on development requirements. To quickly install the corresponding PaddleX plugin by executing the following commands, just replace PaddleXXX with the corresponding plugin name:


```
git clone https://github.com/PaddlePaddle/PaddleX.git
cd PaddleX
pip install -e ".[base]"
paddlex --install PaddleXXX
```

Next, let us see the detailed tutorial for installing PaddleX on Linux system.

1.2.3 Installing PaddleX on Linux(using Docker)

When installing PaddleX on Linux, the official PaddleX Docker image is recommended [2]. Alternatively, use other custom installation methods.

When using the official Docker image, PaddlePaddle, PaddleX (including the wheel package and all plugins) and the corresponding CUDA environment are already preinstalled [1]. You can simply obtain the Docker image and start the container to begin using it. Please note that the official Docker image of PaddleX is different from the official Docker image of the PaddlePaddle framework, as the latter does not come with PaddleX pre-installed [2].

For CPU:

```
docker run --name paddlex -v $PWD:/paddle --shm-size=8g
--network=host -it ccr-2vdh3abv-pub.cnc.bj.baidubce.com/
paddlex/paddlex:paddlex3.0.0-paddlepaddle3.0.0-cpu /bin/bash
```

For GPU, requires a GPU driver version greater than 450.80.02 (Linux) or 452.39 (Windows) [2].

```
docker run --gpus all --name paddlex -v $PWD:/paddle
--shm-size=8g --network=host -it ccr-2vdh3abv-pub.
cnc.bj.baidubce.com/paddlex/paddlex:paddlex3.0.0-
paddlepaddle3.0.0-gpu-cuda11.8-cudnn8.9-trt8.6 /bin/bash
```

With this, PaddlePaddle and Paddlex are installed on a local machine. The next step is data preparation.

2 Model Integration

2.1 Downloading the trained model into local machine

Github Repository -> Paddle Pipeline Repository

The trained model has been uploaded to Google Drive for public access. To use the model, download the entire folder from the provided link Trained_Model. After downloading, the extracted directory will have the following structure:

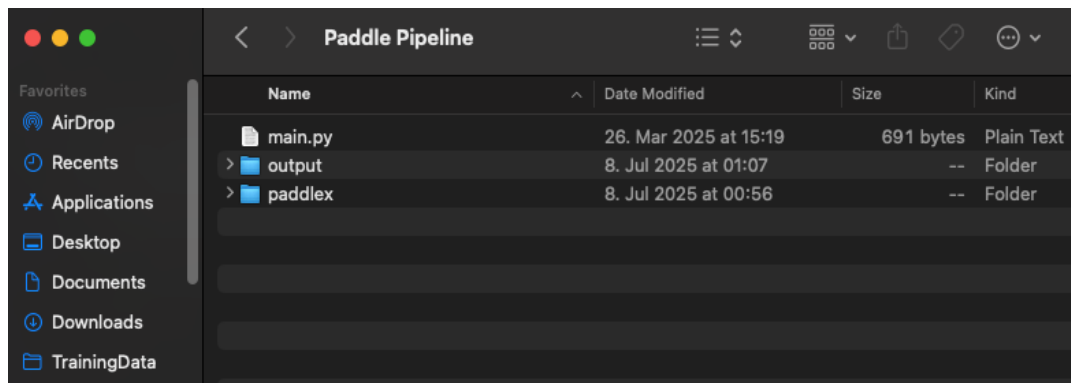


Figure 1: Directory of Trained Model

The pipeline is structured into three key components within the Paddle Pipeline directory for seamless deployment:

1. **main.py** – The core script that executes the entire workflow, integrating data preprocessing, model inference, and output generation.
2. **output/** – Contains the trained model files, including weights (e.g., .pdparams) and metadata, ready for inference.
3. **paddlex/** – Holds the configuration files (e.g., config.yml) defining model architecture, hyperparameters, and training settings.

After downloading the Paddle Pipeline folder, place the document images you wish to test directly inside the Paddle Pipeline/ directory (as shown in the screenshot below).

Name	Date Modified	Size	Kind
1.png	16. Jan 2025 at 18:21	22,7 MB	PNG image
2.png	16. Jan 2025 at 18:21	22,1 MB	PNG image
3.png	16. Jan 2025 at 18:27	22,1 MB	PNG image
4.png	16. Jan 2025 at 18:25	24,5 MB	PNG image
5.png	16. Jan 2025 at 18:23	25,6 MB	PNG image
6.png	16. Jan 2025 at 18:23	22 MB	PNG image
7.png	16. Jan 2025 at 18:20	23,3 MB	PNG image
8.png	16. Jan 2025 at 18:25	23,8 MB	PNG image
main.py	26. Mar 2025 at 15:19	691 bytes	Plain Text
> output	8. Jul 2025 at 01:07	--	Folder
> paddlex	8. Jul 2025 at 00:56	--	Folder

Figure 2: Directory of Trained Model

Ensure all dependencies (listed in README.md) are installed before execution. The model can be loaded using the provided inference script, which supports batch processing for efficient predictions. For any issues, refer to the troubleshooting section in the documentation.

2.2 Running the Pipeline

2.2.1 First Step:

With the model and input files in place, the pipeline is ready for execution. To run the inference process, open a terminal or command prompt, navigate to the Paddle Pipeline directory, and execute the following command:

```
python main.py -c paddlex/configs/modules/layout_detection
                /PP-DocLayout-L.yaml \
-o Global.mode=predict \
-o Predict.model_dir="./output/best_model/inference" \
-o Predict.input="image_name"
```

Upon executing the pipeline, the processed results will be saved in the specified output directory as shown in below image. The system generates two types of files for each input image:

1. **JSON Files:** Contain structured prediction data (e.g. detected text, bounding boxes, confidence scores), Filename format: input_image_name_res.json.
2. **Annotated Images:** Visual overlays of predictions (bounding boxes, labels) for quick verification, Filename format: input_image_name_res.png).

Name	Date Modified	Size	Kind
1_res.json	Today at 15:34	2 KB	Plain Text
1_res.png	Today at 15:34	22,2 MB	PNG image
2_res.json	Today at 15:34	2 KB	Plain Text
2_res.png	Today at 15:34	21,5 MB	PNG image
3_res.json	Today at 15:35	3 KB	Plain Text
3_res.png	Today at 15:35	21,6 MB	PNG image
4_res.json	Today at 15:35	3 KB	Plain Text
4_res.png	Today at 15:35	24 MB	PNG image
5_res.json	Today at 15:35	3 KB	Plain Text
5_res.png	Today at 15:35	25,1 MB	PNG image
6_res.json	Today at 15:35	3 KB	Plain Text
6_res.png	Today at 15:35	21,6 MB	PNG image
7_res.json	Today at 15:35	3 KB	Plain Text
7_res.png	Today at 15:35	22,7 MB	PNG image
8_res.json	Today at 15:36	4 KB	Plain Text
8_res.png	Today at 15:36	23 MB	PNG image
best_model	8. Jul 2025 at 00:58	--	Folder

Figure 3: Directory of Trained Model

2.2.2 Second Step:

To ensure proper document structure in the final output, adhere to the following naming rules for your input files:

1. Front and Back Pages (Mandatory)
 - a) First Page (Cover): Name as front.png (and front.json for its JSON data).
 - b) Last Page (Back Cover): Name as back.png (and back.json for its JSON data).
2. Internal Pages (Sequential Order)
 - a) 2.png, 2.json
 - b) 3.png, 3.json
 - c) ... (and so on)

The Paddle-PyTesseract.py script relies on this naming scheme to preserve document order when compiling pages into a PDF and to tag front/back matter correctly in the TEI XML output (e.g., <front>, <body>, <back>).

Now that the initial predictions are generated, the next step is to extract text from the detected regions in the JSON files. Follow these steps:

1. **Organize JSON Files:** Gather all the output JSON files and their respective png files and place them in a dedicated folder named JSON_files.
2. Execute the Paddle-PyTesseract.py, which will:

- Read each JSON file sequentially.
- Use the bounding-box coordinates to crop and extract text from the original images.
- Generate new JSON files containing the extracted text, saved in a structured format.

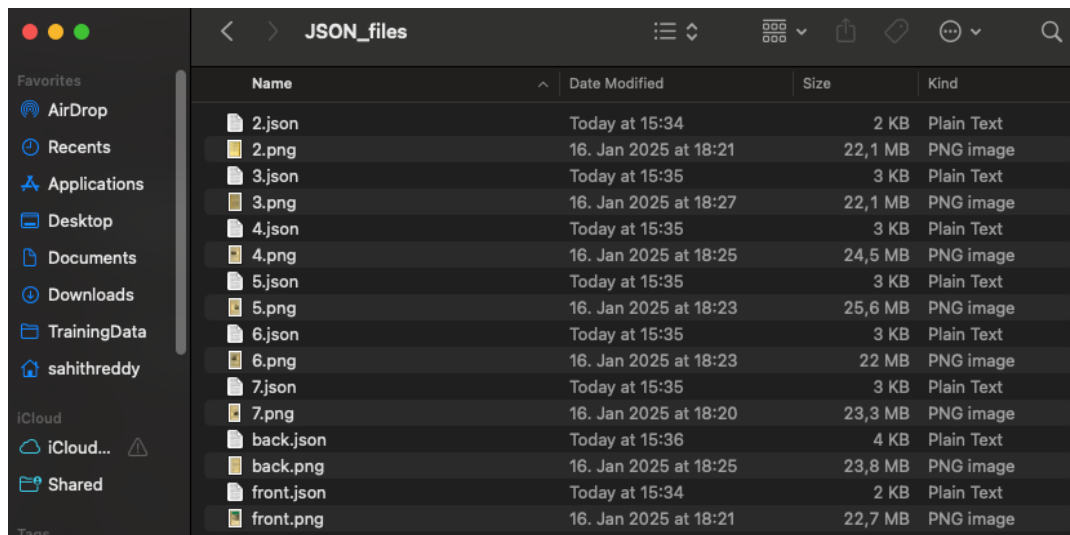


Figure 4: Folder with output JSON and their respective png files

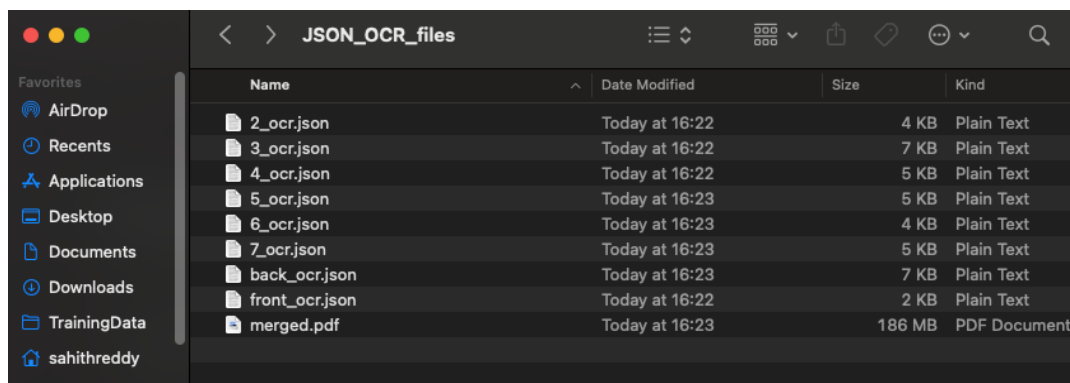


Figure 5: Output folder from Paddle-PyTesseract.py

The output folder created by executing Paddle-PyTesseract.py contains JSON files that has extracted text in it.

2.2.3 Third Step:

With the extracted text stored in JSON files, the final step is to convert them into TEI XML, a standardized format for document encoding. Follow these steps:

1. **Prepare the Input:** Ensure that the JSON files from the JSON_files folder (generated by Paddle-PyTesseract.py) are ready.
2. Execute the Json-Xml.py script, which will:
 - a) Parse each JSON file.
 - b) Map text regions and metadata into TEI XML format.
 - c) Save structured XML files in an output directory (e.g., TEI_XML_Output/).

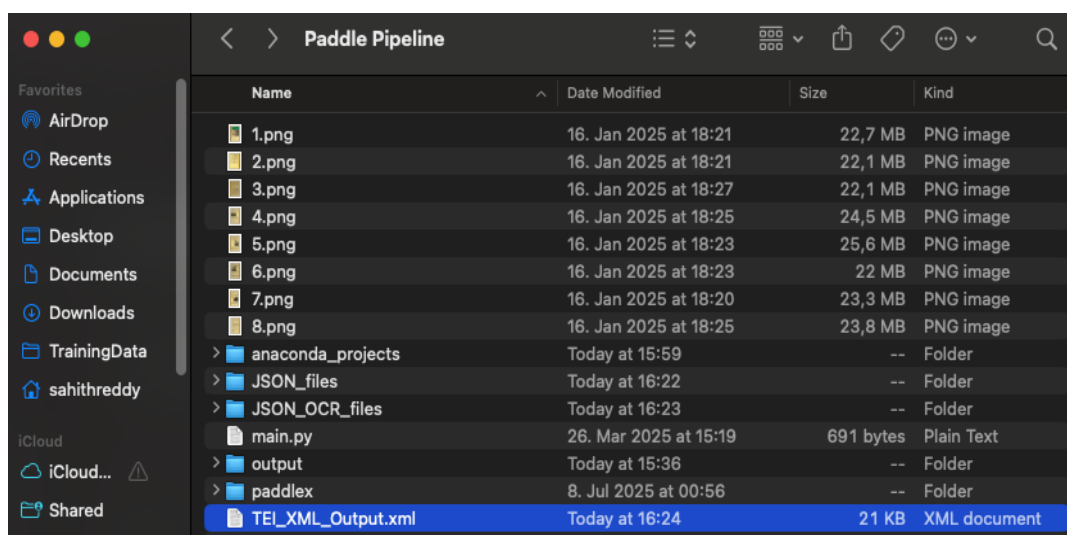


Figure 6: Output of JSON-Xml.py has been downloaded in to the folder

3 Conclusion

By following the outlined workflow—from running the initial PaddleOCR pipeline to organizing JSON files and converting them into structured TEI XML, you can efficiently digitize and standardize document processing. Adherence to the prescribed file naming conventions (front.png, back.png, and sequential numbering) ensures accurate PDF assembly and proper XML tagging for front and back matter. For further optimization, consider scripting the entire process or adding validation checks to maintain consistency across large document batches.

Let us know if you need assistance refining any step or scaling the pipeline for production use!

References

- [1] Github documentation of paddlex.
- [2] Github documentation of paddlex.